

Old way.

```
if (emp instanceof Manager)
{
    Manager objManager = (Manager) emp;
    ;
}
```

Java 14+ → Pattern Matching

X ① if (emp instanceof Manager objManager)
 {
 operations objManager
 }

② switch (emp)
 {
 case Manager objManager → {
 ;
 }

 }
 case Engineer objEngineer →
 {

3.3

wait notify
F notifyAll
Threads synch.
java.lang. Object (Base class to every class in Java)
a) toString →
b) equals → Compare 2 obj
c) hashCode → hash code.
d) clone → copy of obj (deep copy)
e) finalize → finalization
f) getClass() → Reflection
Program
Date
Calculator
ConsoleInput

→ Managed Heap ← JVM → RC++

① Ref. = Ref/Object.

✓ a) Data obj = new Data();
OR.
RC++
RC++

b) Data obj2 = obj;

OR/1.

show (obj);

show (Data arg)

3

OR/1.

Data {
get()
return new Data();

}

RC++

Data value = get();

RC - -

① Data obj = new Data();

a) obj = null;

b) ② obj = new Data(); ✓

→ obj = value;

Data value = new Data();
obj = value;

c) { Data obj = new Data();
...
}

→ Reference Counting MM GC

Garbage Collector

- a) low priority thread
- b) running in background
- c) monitoring RC of an object
- d) destroys the object by reclaiming its memory when the RC = 0
- e) invoking Finalization

Deprecated

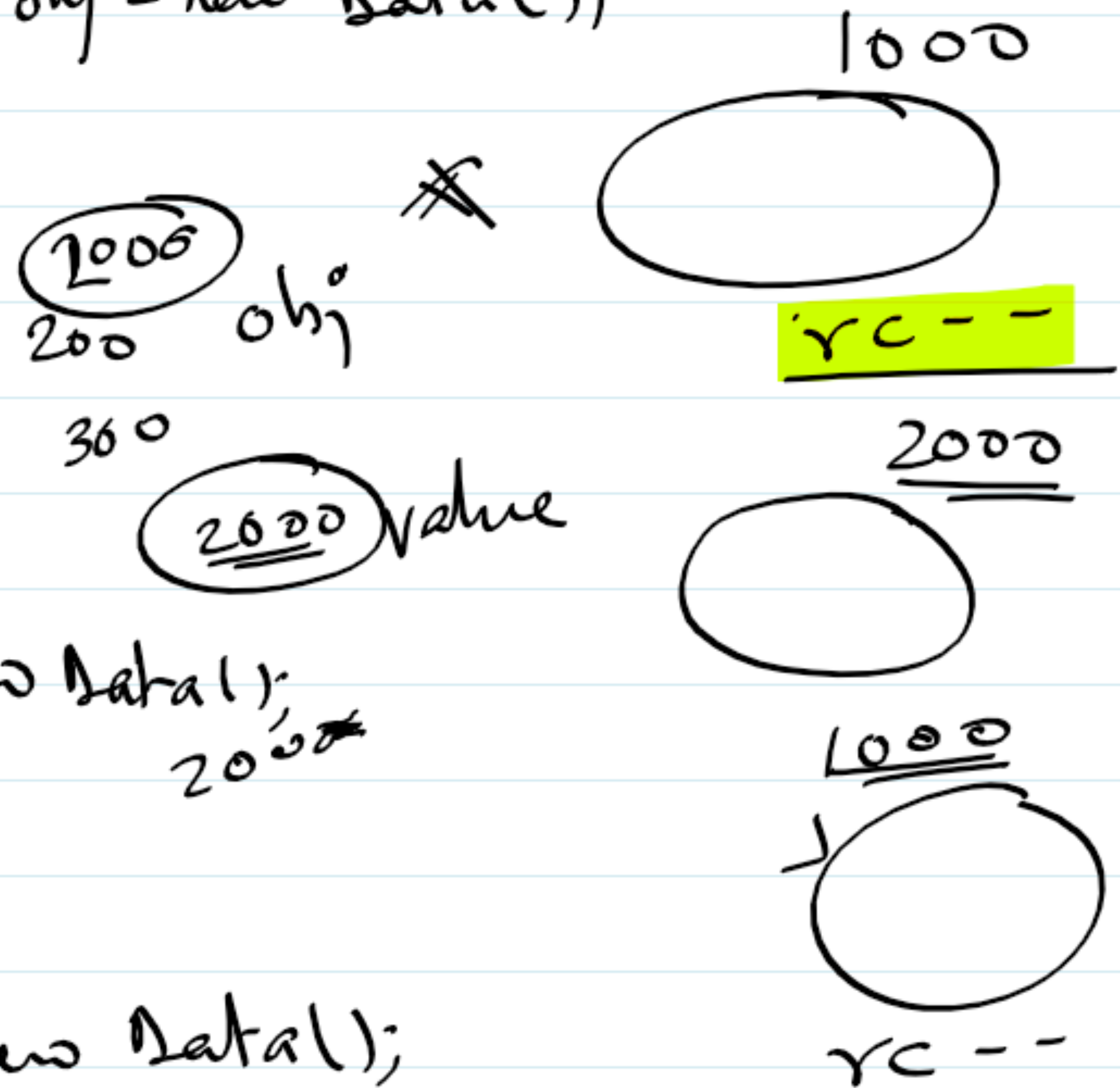
✗ JVM

System.gc()

Request

to increase the priority of
GC Thread

finalize on object
which is about to
be GC



✓	✓	✓	✓
✓	✓	✓	✓
✓	✓	✓	✓
✓	✓		

JVM
 → a) increase pool size
 → b) Reclaim unused memory.

2000
~~200~~

GC
 ① how much is required
 ② GC → can I reclaim

Collectors



Performance
Turning

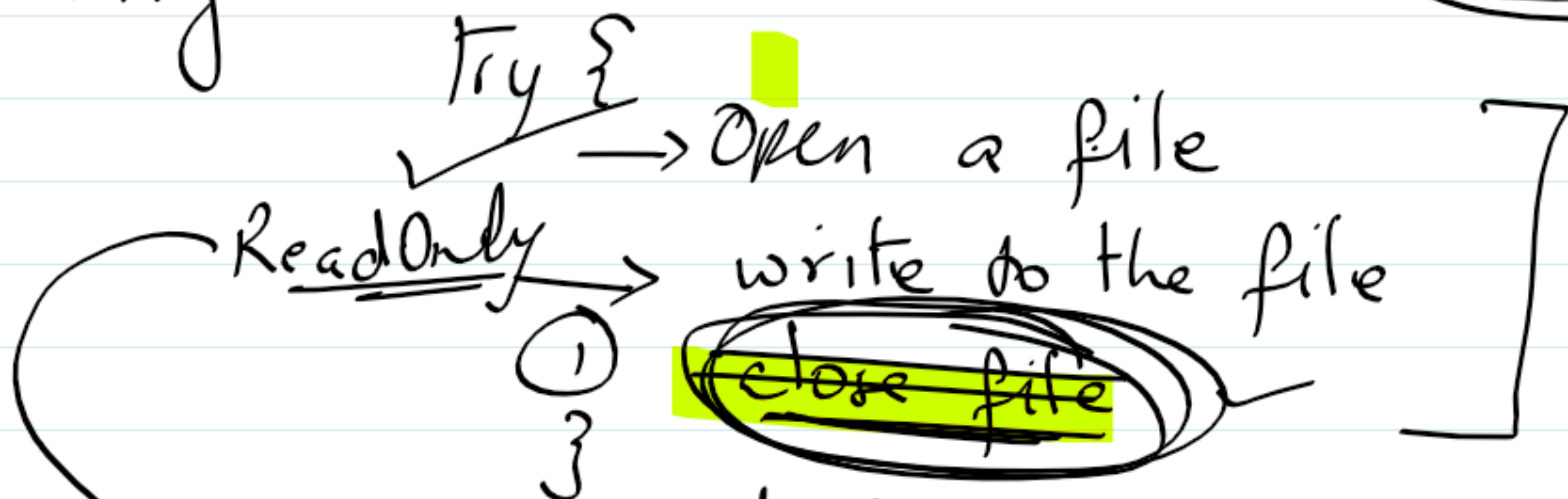
λ

JVM

Exception Handling

- ① try
- ② catch
- ③ throw
- ④ throws
- ⑤ finally

public void reverse(int arr) throws AZOOBE



catch ()

~~② close file~~

finally {

 close file

}

~~catch (Error error)~~

~~throw new Error;~~

~~try {~~

~~int num~~

~~catch ()~~

java.lang.Throwable

Exception

Error

[unchecked]

Runtime Exception

- Null Pointer
- Class Cast
- Array Ind. ---
- ...

[checked]

- IOException
- SQLException
- SocketException
- FileNotFoundException

- OutOfMemory
- BadVersion
- ...

2500+